MODULE 33 · EXAM STRATEGY - ALL DOMAINS

Practice Exam 2

Twenty fresh questions, same domain weighting, no repeats from exam 1.

TYPE Practice exam QUESTIONS 20 PAUSE 9 seconds

admin@sthithapragna.com

Before you start

- **Nine seconds** per question before the answer. Pause if you need longer.
- **Commit out loud.** An uncommitted guess teaches nothing.
- Read the ask, find the deciding clause, check the qualifier.
- Compare your domain breakdown against exam 1, not just the total.
- Two of these questions turn on a number. If you do not know it, note which.

> ON THE EXAM
>
> Comparing domain breakdowns across two exams is more informative than either score alone: a consistent weak domain is a revision target, a one-off dip is noise.

> A Lambda function processes batches of 100 records from an SQS queue. When one record fails, the entire batch returns to the queue and every record is reprocessed, causing duplicate downstream writes.
>
> The team wants only genuinely failed records retried, without reducing the batch size.

What should be configured?

A Set maxReceiveCount to 1 so failed batches go straight to the DLQ

B Enable partial batch responses and return ReportBatchItemFailures from the function

C Switch the queue to FIFO so ordering prevents reprocessing

D Increase the visibility timeout so the batch is not redelivered

◆ PAUSE AND DECIDE

What should be configured?

A maxReceiveCount of 1

Sends the whole batch to the DLQ on first failure, including the 99 records that succeeded. It loses data rather than retrying selectively.

B ReportBatchItemFailures

The function returns the identifiers of failed records only, and Lambda deletes the rest. Exactly the stated requirement, with no batch size change.

C FIFO

FIFO governs ordering and duplicate sends. It does not change how a partially failed batch is handled.

D Longer visibility timeout

Prevents premature redelivery. Here the batch is returned deliberately because it failed, which a longer timeout does not change.

THE SIGNAL

'Only genuinely failed records' with the batch size fixed describes partial batch responses precisely. Everything else trades one problem for another.

A DynamoDB table stores support tickets with customerId as the partition key and ticketId as the sort key. A new requirement is to query a customer's tickets by status, always within that same customer.

The table already exists in production with 40 million items. Reads must be strongly consistent.

What should be created?

A A local secondary index on status

B A global secondary index with status as the partition key

C A global secondary index with customerId as the partition key and status as the sort key

D A new table, because the requirement cannot be met on the existing one

◆ PAUSE AND DECIDE

What should be created?

A **An LSI on status**

An LSI supports strong consistency and the same partition key, which fits perfectly. It can only be created when the table is created, and this table already exists.

B **GSI with status as partition key**

Status has very low cardinality, producing a hot partition, and it would query across all customers rather than within one.

C **GSI on customerId and status**

Wrong on the consistency requirement. GSI reads are eventually consistent only, which the scenario rules out.

D **A new table**

The only way to add an LSI is at table creation, so a new table with the LSI and a migration is the remaining option.

THE SIGNAL

The trap is that A is the correct index type and cannot be added. LSIs are creation-time only, which turns this into a migration question.

A finance team reports that an S3 bucket's storage bill is 35% higher than the total size of the objects it contains. A mobile application uploads large files to the bucket and frequently loses connectivity mid-upload.

Listing the bucket accounts only for the expected object storage. Versioning is not enabled.

What should be configured?

A A lifecycle rule with AbortIncompleteMultipartUpload

B A lifecycle rule expiring noncurrent versions

C A lifecycle transition to S3 Standard-Infrequent Access

D S3 Transfer Acceleration to reduce failed uploads

◆ PAUSE AND DECIDE

What should be configured?

A AbortIncompleteMultipartUpload

Interrupted uploads leave parts that are billed but invisible to a listing, which explains storage that listing cannot account for. The rule removes them and prevents recurrence.

B Expire noncurrent versions

The scenario states versioning is not enabled, so there are no noncurrent versions to expire.

C Transition to Standard-IA

Reduces cost per gigabyte and leaves the invisible storage in place. It treats the symptom while the leak keeps growing.

D Transfer Acceleration

Speeds uploads and costs more, not less. It also does nothing about parts already accumulated.

THE SIGNAL

Three clues converge: storage a listing cannot see, large uploads, and dropped connections. Only incomplete multipart parts fit all three.

> A company sells tiered access to a public API: free at 10 requests per second and 10,000 per month, and paid at 500 per second with no monthly cap. Excess requests must be rejected before the backend is invoked, and per-customer usage must be reportable.
>
> Customers already authenticate with a Cognito user pool.

What should be built?

A An HTTP API with a JWT authoriser and per-route throttling

B A Lambda authoriser counting requests per customer in DynamoDB

C A REST API with usage plans and API keys, alongside the Cognito authoriser

D AWS WAF rate-based rules, one per tier

◆ PAUSE AND DECIDE

What should be built?

A HTTP API with route throttling

HTTP APIs have no API keys and no usage plans, so per-customer quotas are impossible. Route throttling applies to everyone equally.

B Lambda authoriser counting

Rebuilds usage plans by hand, invokes a function on every request including rejected ones, and adds a DynamoDB write to the request path.

C REST API with usage plans

Usage plans express rate, burst and monthly quota per key, reject at the gateway, and produce per-key usage reports. Cognito still handles authentication.

D WAF rate-based rules

WAF rate rules work on IP addresses over five-minute windows. They cannot express a per-customer monthly quota.

THE SIGNAL

The Cognito detail is a distractor. Authentication and metering are separate jobs, and this API needs both.

An order event must reach three independent consumers: fulfilment, analytics and a partner webhook. Each processes at a different speed. The analytics consumer is regularly offline for maintenance for up to two hours.

No event may be lost for any consumer, and adding a fourth consumer must not require changing the publisher.

What architecture should be used?

A One SQS queue polled by all three consumers

B An SNS topic with all three Lambda functions subscribed directly

C The publisher calling each consumer in turn

D An SNS topic with a separate SQS queue subscribed for each consumer

◆ PAUSE AND DECIDE

What architecture should be used?

A One shared queue

A queue message is consumed by one consumer. Three consumers polling one queue means each event is processed once, by whichever gets there first.

B SNS direct to Lambda

The strongest distractor. SNS does not store messages, so two hours of analytics downtime loses those events once retries are exhausted.

C Direct calls

Recouples everything. The publisher now knows all three consumers, and a slow one blocks order publication.

D SNS fan-out to per-consumer queues

Each consumer gets a durable backlog, so the analytics outage is survivable, and a fourth consumer means a fourth queue with no publisher change.

THE SIGNAL

'Offline for two hours' and 'no event may be lost' together demand durability. SNS provides the fan-out; only the queues provide the backlog.

> A workflow must process roughly 8 million objects listed in an S3 bucket, running an ECS task on each. Corrupt objects should not stop the run, and the team needs a report of which failed.
>
> The object list is far too large to pass as workflow input.

How should this be orchestrated?

A A Distributed Map state reading the object list from S3, with a tolerated failure threshold

B An inline Map state over an array of all object keys

C A Parallel state with one branch per object

D An Express workflow triggered once per object by S3 event notifications

◆ PAUSE AND DECIDE

How should this be orchestrated?

A Distributed Map from S3

Reads items directly from S3 so nothing passes through state, scales to 10,000 child executions, and the tolerated failure threshold handles corrupt objects.

B Inline Map

The array would have to fit in the 256 KB state payload. Eight million keys is orders of magnitude past that.

C Parallel with one branch per object

Parallel branches are defined statically in the template. You cannot write eight million, and any failure fails the state.

D Express per object

Gives no aggregate view of the run and no failure report, and loses the tolerated failure behaviour entirely.

THE SIGNAL

'Far too large to pass as workflow input' names the 256 KB state payload limit without stating it. Only Distributed Map reads items from S3 instead.

A platform team wants application developers to create IAM roles for their own Lambda functions without waiting for approval. Security requires that no role a developer creates can ever exceed read access to S3 and write access to one DynamoDB table.

Developers must not be able to escalate to administrator, directly or indirectly.

How should this be implemented?

A Grant iam:CreateRole and review each created role manually

B Apply a service control policy to the account limiting all principals

C Grant iam:CreateRole with a condition requiring a permissions boundary on every role created

D Pre-create a fixed set of roles and let developers select one

◆ PAUSE AND DECIDE

How should this be implemented?

A CreateRole plus manual review

Manual review is not a control. A developer can create and use a privileged role before anyone looks at it.

B SCP on the account

Would cap every principal in the account, including the platform team and production workloads. Far too blunt for delegation.

C CreateRole conditioned on a boundary

Effective permissions are the intersection of the identity policy and the boundary, so a role carrying the boundary cannot exceed it whatever policy is attached.

D Fixed set of roles

Removes the escalation risk by removing the delegation, which reintroduces the approval wait the scenario is trying to eliminate.

THE SIGNAL

Delegation plus a hard ceiling on identities the developers create themselves is the textbook permissions boundary case.

A multi-tenant SaaS application uses a Cognito user pool behind an API Gateway Cognito authoriser. Every Lambda function currently queries DynamoDB on each request to discover the caller's tenant, adding latency and cost.

Tenant membership changes rarely. The value must reach the backend without a lookup and must be tamper-proof.

What should be implemented?

A Have the client send the tenant ID in a custom header

B Add a pre token generation trigger that injects the tenant ID as a claim

C Cache the tenant lookup in ElastiCache keyed by user ID

D Replace the Cognito authoriser with a Lambda authoriser that caches the lookup

◆ PAUSE AND DECIDE

What should be implemented?

A Client-supplied header

A client-controlled value is not tamper-proof. Any caller could claim another tenant, which is a cross-tenant data breach.

B Pre token generation trigger

The tenant becomes a signed claim in the token. No lookup at all, and the signature guarantees integrity. Rare membership changes make token lifetime acceptable.

C ElastiCache lookup

Faster than DynamoDB, and still a lookup on every request, plus a cache to run and invalidate.

D Lambda authoriser with caching

Works, and adds a function invocation and its own cache TTL to manage. More moving parts than putting the value in the token.

THE SIGNAL

'Tamper-proof' is the deciding word. A signed token claim cannot be altered by the client; a header can.

A team manages 4,000 configuration values across three environments: endpoints, timeouts, feature toggles and a handful of static third-party API keys. None of them rotates.

Applications must fetch all settings for their environment in one call, and the current cost of one Secrets Manager secret per value is unacceptable.

Where should these values live?

A Consolidate into a smaller number of large Secrets Manager secrets

B Environment variables set at deployment time by the pipeline

C Parameter Store advanced tier, for the higher parameter limit

D Parameter Store standard tier with a path hierarchy per environment, SecureString for the keys

◆ PAUSE AND DECIDE

Where should these values live?

A Fewer, larger secrets

Reduces cost by making configuration harder to change. Updating one timeout means rewriting a large JSON blob.

B Environment variables

Changing any value now requires a deployment, and the API keys end up readable by anyone who can describe the function.

C Advanced tier

Advanced costs about 5 cents per parameter per month, so 4,000 parameters is roughly 2,400 dollars a year. Worse than the problem.

D Standard tier with hierarchy

4,000 is within the 10,000 standard limit, standard is free, GetParametersByPath satisfies the one-call requirement, and SecureString covers the keys.

THE SIGNAL

Option C is the trap: 'thousands of parameters' sounds like it needs the higher tier, and 4,000 is well inside the standard limit of 10,000.

A company runs 250 servers in its own data centre. Each writes backups to Amazon S3 using an IAM user's access keys stored in a configuration file, unrotated for two years.

Security requires temporary, automatically rotating credentials. The servers cannot move to EC2, and the company operates its own certificate authority.

What should the company implement?

A IAM Roles Anywhere, with the existing CA registered as a trust anchor

B Store the access keys in AWS Secrets Manager and fetch them at startup

C Rotate the access keys automatically with a scheduled Lambda function

D Run an EC2 proxy that the on-premises servers call

◆ PAUSE AND DECIDE

What should the company implement?

A IAM Roles Anywhere

Exchanges an X.509 certificate for temporary credentials from STS. No stored keys, automatic rotation, and the CA already exists.

B Keys in Secrets Manager

Circular. Reading Secrets Manager requires AWS credentials, so the server still needs a stored credential to start.

C Automated key rotation

Improves a bad pattern rather than replacing it. The keys are still long-lived between rotations and still in a config file.

D EC2 proxy

Every backup byte now crosses a server you must run, scale and secure, to avoid a credential problem with a direct solution.

THE SIGNAL

Option B's circularity is the useful lesson: you cannot use an AWS service to bootstrap AWS credentials. The CA detail is the planted clue.

A Lambda function must encrypt 60 MB report files before writing them to Amazon S3, using a customer managed KMS key so that key use is auditable and revocable.

A first attempt called the KMS Encrypt API with the file contents and failed.

How should the encryption be implemented?

A Split the file into 4 KB chunks and call Encrypt on each

B Call GenerateDataKey, encrypt the file locally with the plaintext key, and store the encrypted data key alongside

C Use an asymmetric KMS key, which has no payload limit

D Request a quota increase for the KMS Encrypt payload size

◆ PAUSE AND DECIDE

How should the encryption be implemented?

A **4 KB chunks**

60 MB is about 15,000 chunks, so 15,000 KMS calls per file. Slow, expensive, and it would throttle the account.

B **Envelope encryption**

One KMS call per file, encryption happens locally at full speed, and the stored encrypted data key is useless without KMS.

C **Asymmetric key**

Asymmetric keys have a smaller payload limit, not a larger one, and are far slower for bulk data.

D **Quota increase**

4 KB is a hard API limit, not an adjustable quota. No request changes it.

THE SIGNAL

Every wrong answer tries to make KMS encrypt the data. The design is that it never does: it encrypts the key that encrypts the data.

A team updated a CloudFormation stack to change a property on an RDS instance. CloudFormation replaced the instance, creating a new empty database and deleting the original. Several hours of data were lost.

The team wants certainty this cannot recur, and wants to see the impact of a change before applying it.

What should the team adopt?

A Enable termination protection on the stack

B Run drift detection before each deployment

C Create a change set before every update, and set both DeletionPolicy and UpdateReplacePolicy on the database

D Disable automatic rollback so failed updates leave resources in place

◆ PAUSE AND DECIDE

What should the team adopt?

A **Termination protection**

Protects against deleting the whole stack. This was an update, and the stack was never deleted.

B **Drift detection**

Finds changes made outside CloudFormation. This change came through CloudFormation, so there was no drift to find.

C **Change set plus both policies**

The change set shows Replacement: True in advance, and UpdateReplacePolicy: Snapshot preserves the old database even if replacement happens.

D **Disable rollback**

Affects what happens after a failure. This update succeeded exactly as CloudFormation intended, which is the problem.

THE SIGNAL

UpdateReplacePolicy is the crucial half. This was a replacement during an update, so DeletionPolicy alone would not have saved the data.

A security baseline stack containing IAM roles and Config rules must be deployed into 80 AWS accounts across 3 Regions. New accounts join the organisation most weeks and must receive the baseline without manual action.

All accounts belong to one AWS Organization.

How should this be deployed?

A. A service-managed StackSet targeting the OU, with automatic deployment enabled

B. A nested stack in the management account with one child per account

C. One stack in the management account exporting values the others import

D. A pipeline assuming a role in each account and running create-stack in a loop

◆ PAUSE AND DECIDE

How should this be deployed?

A Service-managed StackSet

One template across accounts and Regions, and automatic deployment adds new accounts to the OU without any action. Exactly the requirement.

B Nested stacks

Nested stacks are created in the same account and Region as the parent. They cannot span accounts at all.

C Exports and imports

Exports are Regional and same-account. Fn::ImportValue cannot read an export from another account.

D Pipeline looping create-stack

Rebuilds StackSets by hand, with 240 stacks to track, no failure tolerance and its own code to maintain.

THE SIGNAL

'New accounts must receive it without manual action' selects service-managed over self-managed. Only the Organizations model does automatic deployment.

An ECS service must run a full integration suite against new tasks using real HTTP requests before any production traffic reaches them. The tests take about five minutes.

Production traffic must be entirely unaffected while the tests run, and a failing test must abort the deployment.

How should this be configured?

A Run the tests in a CodeBuild stage before the deploy stage

B Use a Linear10PercentEvery1Minute configuration so failures affect few users

C Add a ValidateService hook that runs the integration suite

D Configure a test listener on a second port, with an AfterAllowTestTraffic hook running the tests

◆ PAUSE AND DECIDE

How should this be configured?

A CodeBuild stage before deploy

Tests a build artifact, not the deployed service. It cannot make real HTTP requests to tasks that do not exist yet.

B Linear traffic shifting

Limits the blast radius rather than preventing it. Production traffic reaches the new tasks immediately, which the scenario forbids.

C ValidateService hook

An EC2-only hook. It does not exist for ECS deployments.

D Test listener plus AfterAllowTestTraffic

The test listener serves the replacement task set on a separate port, so real requests reach it while production traffic does not. A failed hook aborts the deployment.

THE SIGNAL

'Real HTTP requests' plus 'production entirely unaffected' describes the test listener precisely. Option C is a hook from another platform.

> A developer iterating on Lambda business logic waits about 80 seconds for sam build and sam deploy on every change. They make dozens of changes an hour and must test against real DynamoDB and SQS resources in a development account.
>
> Local emulation is insufficient because the logic depends on real service behaviour.

What should the developer use?

A sam local start-api with the SDK pointed at the development account

B sam sync --watch, accepting that it bypasses CloudFormation during development

C A script calling the Lambda UpdateFunctionCode API directly

D Reduce the deployment package size to shorten the deploy

◆ PAUSE AND DECIDE

What should the developer use?

A sam local start-api

The scenario rules this out: local emulation is explicitly insufficient because the logic depends on real service behaviour.

B sam sync --watch

Purpose-built for this: pushes code changes in seconds, watches files automatically, and the CloudFormation bypass is acceptable in a development account.

C UpdateFunctionCode script

Hand-builds what sam sync does, without dependency handling or multi-function support, and with the same drift downside.

D Smaller package

Most of the 80 seconds is change set creation and execution, not upload. A smaller package barely moves it.

THE SIGNAL

'Development account' is what makes B acceptable. The same answer would be wrong for production, because bypassing CloudFormation is the mechanism.

A company publishes internal npm packages under a scoped name. A security audit warns that an attacker could publish a package with the same name to the public registry at a higher version, and builds might resolve to it.

Builds must still install genuine public dependencies normally.

What should the company implement?

A Pin every dependency to an exact version in the lockfile

B Remove the external connection and vendor all public packages manually

C CodeArtifact with an external connection, and package origin controls blocking upstream versions of internal names

D Run npm audit in a build phase and fail on any finding

◆ PAUSE AND DECIDE

What should the company implement?

A Pin exact versions

Helps, and breaks the moment anyone adds or upgrades a dependency. It is a discipline rather than a control.

B Vendor everything

Eliminates the risk by eliminating the convenience, and contradicts the requirement to install genuine public dependencies.

C CodeArtifact origin controls

The named defence for dependency confusion. Internal names can never be fetched from upstream, while genuine public packages resolve normally.

D npm audit

Reports known vulnerabilities in real packages. A malicious package impersonating an internal name is not a known vulnerability.

THE SIGNAL

This attack has a name, dependency confusion, and a specific AWS answer. Recognising the shape is what makes the question quick.

A central security team needs every application log event from 150 Lambda functions across 8 accounts searchable in one OpenSearch domain within a minute of being written.

They do not need alarms on the log content, and the volume is high.

What should be configured?

A Subscription filters on the log groups, delivering to Amazon Data Firehose and on to OpenSearch

B A metric filter on each log group publishing to a shared namespace

C A nightly export task from each log group to S3, loaded into OpenSearch

D Move all log groups to the Infrequent Access class and query with OpenSearch SQL

◆ PAUSE AND DECIDE

What should be configured?

A Subscription filters to Firehose

Near real-time delivery of the events themselves, cross-account destinations are supported, and Firehose handles buffering at high volume.

B Metric filters

Produces counts, not events. The team needs to read and correlate the log lines themselves.

C Nightly export

The requirement is within a minute. A nightly job is off by three orders of magnitude.

D Infrequent Access plus SQL

Leaves logs in each account rather than centralising them, and IA does not support subscription filters at all.

THE SIGNAL

'Searchable' plus 'within a minute' means the events, delivered in near real time. That is the subscription filter definition.

An alarm watches the Errors metric of a Lambda function invoked a few times an hour. During a recent incident the function failed on every invocation for four hours, and the alarm never left INSUFFICIENT_DATA.

The alarm uses a one-minute period, one datapoint to alarm, and default missing data treatment.

What is the correct fix?

A Set missing data treatment to breaching

B Widen the period to match the invocation rate, and alarm on an error rate using metric math

C Switch the metric to high resolution

D Create a composite alarm combining Errors and Duration

◆ PAUSE AND DECIDE

What is the correct fix?

A Missing data as breaching

The tempting one. It would fire during the incident and also fire every quiet minute, because a function invoked hourly is missing data most of the time.

B Widen the period, alarm on error rate

A period long enough to contain invocations produces datapoints, and an error rate from Errors divided by Invocations is meaningful at any volume.

C High resolution

Publishes datapoints more often only when there is data. A function not invoked still produces nothing.

D Composite alarm

Combines alarms that already work. Both underlying metrics have the same gap, so combining them changes nothing.

THE SIGNAL

The mismatch is a one-minute period against an hourly invocation rate. Option A treats the symptom and would page the on-call engineer all night.

> A Lambda function processing orders at 3,000 invocations per second must record order value and processing duration per customer tier, alarm on average value by tier, and allow investigation of individual slow orders by customer ID.
>
> Latency is critical.

How should the instrumentation be done?

A Call PutMetricData once per invocation with customer ID and tier as dimensions

B Write plain log lines and create a metric filter per tier

C Call PutMetricData in a background thread after returning the response

D Emit Embedded Metric Format log lines, with tier as a dimension and customer ID as a property

◆ PAUSE AND DECIDE

How should the instrumentation be done?

A PutMetricData with customer as a dimension

Two problems: a synchronous call on every invocation at 3,000 per second, and customer ID as a dimension creates a billable metric per customer.

B Metric filters per tier

Workable for counts, and it means a filter per tier to maintain, and extracting numeric values like order value is awkward.

C Background thread

Lambda freezes the execution environment after the handler returns, so background work may never run.

D EMF, tier as dimension, customer as property

No API call so no added latency, tier is a low-cardinality dimension you can alarm on, and customer ID stays queryable without becoming a metric.

THE SIGNAL

The dimension-versus-property distinction is the whole question. Tier has a handful of values; customer ID has millions.

> Lambda functions are attached to a VPC to reach a private Aurora cluster. They also read and write Amazon S3 and DynamoDB heavily. The bill shows a large NAT gateway charge with high data processing costs.
>
> The functions call nothing outside AWS.

How should the networking cost be reduced?

A Remove the functions from the VPC entirely

B Move the NAT gateway to a single Availability Zone

C Add gateway VPC endpoints for S3 and DynamoDB, and remove the NAT gateway

D Replace the NAT gateway with a NAT instance on a small EC2 host

◆ PAUSE AND DECIDE

How should the networking cost be reduced?

A Leave the VPC

The functions need a private Aurora cluster, which requires VPC attachment. This removes the cost by breaking the requirement.

B Single-AZ NAT gateway

Reduces the fixed charge and keeps all the per-GB processing. It also creates an availability single point of failure.

C Gateway endpoints, drop the NAT

Gateway endpoints for S3 and DynamoDB are free with no per-GB charge, and with no external calls the NAT gateway is unnecessary entirely.

D NAT instance

Cheaper per hour and now something you patch, scale and monitor, and it still processes every byte.

THE SIGNAL

'Call nothing outside AWS' is what allows removing the NAT gateway altogether rather than merely making it cheaper.

Scoring this exam

- **17 or more:** comfortable. Move to practice exam 3.
- **14 to 16:** passing territory. Review where you dropped marks.
- **Below 14:** revisit those modules before booking.
- **Compare domain breakdowns.** A domain weak in both exams is a real gap.
- A domain weak in only one is more likely a sampling artefact than a knowledge gap.

> ON THE EXAM
>
> Two exams give you a pattern rather than a number. The questions you got right without being able to justify are still the ones to revisit.